

OIM7502-01

NFL GAME PREDICTION & SALARY ANALYSIS - ML MODELS FOR 2024 SEASON

BY ALAN, MARIA, JJ AND VANS



INTRODUCTION

The NFL (National Football League) is the top American football league with 32 teams and highly unpredictable outcomes (“any given Sunday”).

01 GOAL

Use machine learning to predict NFL game results using only pre-game historical data.

02 APPROACH

Train models on 2022–2023 data and test on 2024 to simulate real forecasting.

03 MODELS

Logistic Regression, Random Forest, KNN, and a Stacked Ensemble.

DATASET AND STRUCTURE

- 1,708 NFL games (2022–2024) → expanded into 3,416 team-rows
- 113 original columns → engineered down to 20 predictive features
- Train set: 2022–23 (4,344 rows)
- Test set: 2024 (2,176 rows)
- Only pre-kickoff information used (lagged features)

season	week	game_id	team	opponent	points_for	points_against	margin	result	season_type
2022	1	2022_01_KC_ARI	ARI	KC	21	44	-23	L	REG
2022	1	2022_01_NO_ATL	ATL	NO	26	27	-1	L	REG
2022	1	2022_01_BAL_NYJ	BAL	NYJ	24	9	15	W	REG
2022	1	2022_01_BUF_LA	BUF	LA	31	10	21	W	REG
2022	1	2022_01_CLE_CAR	CAR	CLE	24	26	-2	L	REG
2022	1	2022_01_SF_CHI	CHI	SF	19	10	9	W	REG
2022	1	2022_01_PIT_CIN	CIN	PIT	20	23	-3	L	REG
2022	1	2022_01_CLE_CAR	CLE	CAR	26	24	2	W	REG
2022	1	2022_01_TB_DAL	DAL	TB	3	19	-16	L	REG
2022	1	2022_01_DEN_SEA	DEN	SEA	16	17	-1	L	REG
2022	1	2022_01_PHI_DET	DET	PHI	35	38	-3	L	REG
2022	1	2022_01_GB_MIN	GB	MIN	7	23	-16	L	REG
2022	1	2022_01_IND_HOU	HOU	IND	20	20	0	T	REG
2022	1	2022_01_IND_HOU	IND	HOU	20	20	0	T	REG
2022	1	2022_01_JAX_WAS	JAX	WAS	22	28	-6	L	REG
2022	1	2022_01_KC_ARI	KC	ARI	44	21	23	W	REG
2022	1	2022_01_BUF_LA	LA	BUF	10	31	-21	L	REG
2022	1	2022_01_LV_LAC	LAC	LV	24	19	5	W	REG
2022	1	2022_01_LV_LAC	LV	LAC	19	24	-5	L	REG

Data Pipeline:

[1,708 games] → [3,416 rows] → [20 features] → [4,344 train / 2,176 test]

2022-2024 2 perspectives Engineered Time-based split

AVOIDING DATA LEAKAGE (CRITICAL STEP IN OUR MODEL)

- **We used a time-based split:** Train on 2022–2023, test on 2024, ensuring the model only learns from past games to predict future ones.
- **Applied `.shift(1)` for all historical features:** Rolling averages (win%, EPA, turnovers) were shifted down by one game so stats never include the game being predicted.
- **Removed outcome-related variables:** Dropped post-game data (points scored, margin, result) to prevent leakage.
- **Created matchup features fairly:** Team vs opponent stats were merged only after all features were lagged.

Why this matters: Prevents the model from “cheating,” keeps predictions realistic, and boosts generalization, enabling our LR model to achieve 72.9% true forecasting accuracy.

What It Does:

python

```
df['win_pct'] = df.groupby('team')['result'].shift(1).expanding().mean()
```

Step-by-Step Example:

Team: MIN

Game 1: Win (result=1) → win_pct = NaN (no history yet)

Game 2: Win (result=1) → win_pct = 1.00 (uses only Game 1)

Game 3: Loss (result=0) → win_pct = 1.00 (uses Games 1-2)

Game 4: Win (result=1) → win_pct = 0.67 (uses Games 1-3)

The Magic of `.shift(1)`: It moves each value down by one row, ensuring that Game N's features only use information from Games 1 through N-1.

MODELS WE BUILT

LOGISTIC REGRESSION

- Linear combination of features
- Produces win probabilities

RANDOM FOREST

- 100 trees
- Captures nonlinear interactions

KNN

- Finds 11 most similar historical games
- Struggles due to curse of dimensionality

STACKED MODEL

- Meta-model combining LR + RF + KNN
- Underperformed due to heavy reliance on weak KNN (coef = 10.472)

MODEL PERFORMANCE COMPARISON

Logistic Regression performed with Random Forest close behind.
KNN showed the weakest results, and the stacked model failed due to over-reliance on KNN.
Simpler, well-engineered models can outperform complex ones.

Model	Accuracy	Precision	Recall	F1-Score	Rank Correlation (ρ)	P-Value
Logistic Regression	72.9%	0.73	0.73	0.73	0.800	< 0.001
Random Forest	71.5%	0.71-0.72	0.71-0.72	0.71-0.72	0.830	< 0.001
KNN	61.2%	0.61	0.61-0.62	0.61	0.601	< 0.001
Stacked Ensemble	62.5%	0.62-0.63	0.62-0.63	0.62-0.63	0.666	< 0.001
Vegas Oddsmakers	65-70%	-	-	-	-	-
ESPN FPI	~68%	-	-	-	-	-

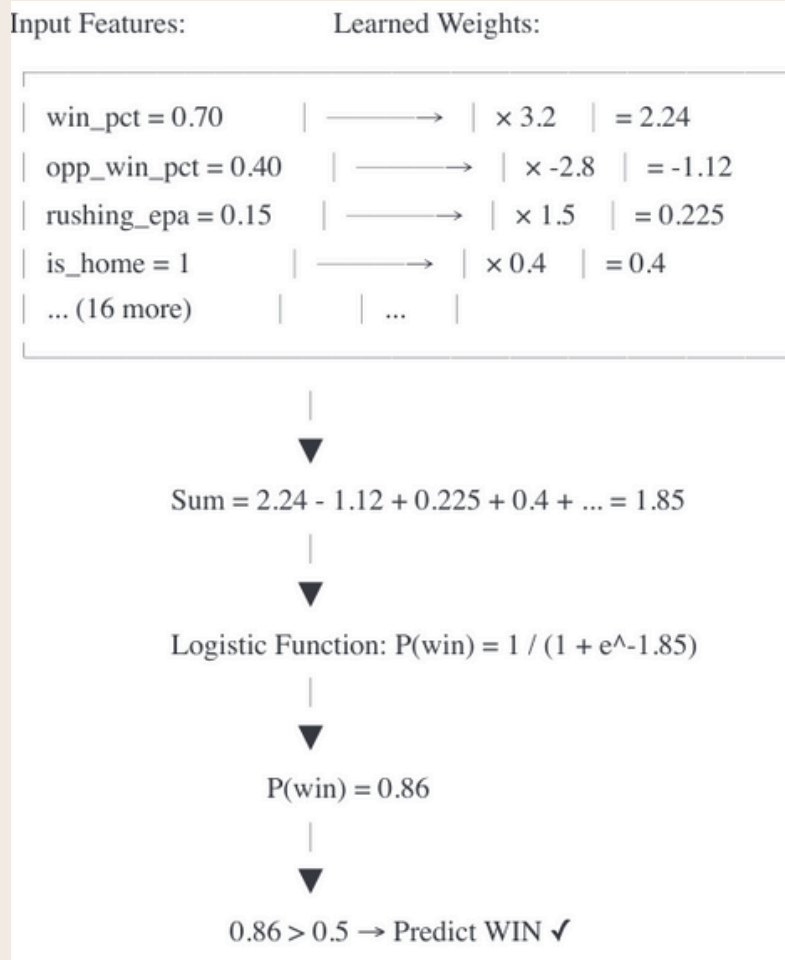
Winner: Logistic Regression for accuracy, Random Forest for ranking

WHY LOGISTIC REGRESSION WON

- NFL relationships are mostly additive (rushing + passing + home field)
- Well-engineered features align with linear modeling
- Regularization prevents overfitting
- Symmetric confusion matrix ($\approx 73\%$ precision/recall for wins & losses) shows no bias

	Predicted Loss	Predicted Win	
Actual Loss	791	297	72.7% recall
Actual Win	294	794	73.0% recall
	72.9%	72.8%	
	precision	precision	

What It Does: Logistic Regression learns a weighted combination of the 20 features to predict the probability of a team winning. It creates a "scoring formula" where each statistic (rushing EPA, win percentage, etc.) gets multiplied by a coefficient, then summed up and converted to a probability between 0 and 1



RANDOM FOREST: BEST AT RANKING TEAMS

- Accuracy slightly below LR (71.5%)
- Best Spearman rank correlation ($\rho = 0.830$)
- Useful for ranking team strength, not individual game prediction

What It Does: Random Forest builds 100 separate decision trees (`n_estimators=100`), each trained on a random subset of data and a random subset of features. For predictions, all 100 trees vote, and the majority wins.

	Predicted Loss	Predicted Win	
Actual Loss	785	303	72.1% recall
Actual Win	317	771	70.9% recall
	71.2%	71.8%	
	precision	precision	

Rank	Feature	Importance	Interpretation
1	win_pct	11.3%	Team historical win rate
2	opp_win_pct	10.6%	Opponent historical win rate
3	rel_rushing_epa_team_avg	4.4%	Rushing efficiency advantage
4	rel_passing_interceptions_team_avg	2.2%	Turnover differential
5	is_home	1.8%	Home field advantage

WHY KNN FAILED

- With 20 features, all games appear equally distant
- KNN can't find truly "similar" past games
- NFL outcomes have high inherent variance
- Result: 61.2% accuracy and lowest feature importance scores

What It Does: KNN makes predictions by finding the K most similar historical games in the training data and taking a majority vote. It's a "lazy learner" as it doesn't build a model during training, it just stores all 4,344 training examples and compares new games to them.

	Predicted Loss	Predicted Win	
Actual Loss	670	418	61.6% recall
Actual Win	425	663	60.9% recall
Precision:	61.2%	61.4%	
Overall Accuracy:		61.2%	

Example Failure Mode:

Test Game: MIN vs CHI, win_pct=0.7, rushing_EPA=0.2, is_home=1
Nearest Neighbor #1: GB vs DET 2023, similar stats → Won
Nearest Neighbor #2: TB vs NO 2022, similar stats → Lost
...
Nearest Neighbor #11: BUF vs MIA 2023, similar stats → Lost
Vote: 6 Lost, 5 Won → Predict Loss (but MIN actually won)

FEATURE IMPORTANCE: WHAT ACTUALLY PREDICTS NFL WINS

Logistic Regression

Rank	Feature	Importance	Interpretation
1	opp_win_pct	11.6%	Opponent historical win rate
2	win_pct	11.5%	Team historical win rate
3	rel_rushing_epa_team_avg	4.6%	Rushing efficiency advantage
4	rel_passing_interceptions_team_avg	1.7%	Turnover differential
5	is_home	1.6%	Home field advantage
6	rel_receiving_epa_team_avg	1.5%	Receiving efficiency
7	rel_turnovers_team_avg	1.3%	Turnover rate
8	rel_def_interceptions_team_avg	1.0%	Defensive takeaways
9	rel_def_sacks_team_avg	0.7%	Pass rush effectiveness
10	rel_receiving_yards_team_avg	0.6%	Receiving volume

Key Insight: Win percentage features (1 & 2) account for 23.1% of predictive power combined

Win percentage features dominate the model, with efficiency and turnover metrics adding smaller but meaningful contributions.

This confirms a logical, stable feature set for predicting game outcomes.

SALARY ANALYSIS

- Correlation r between salary & win% = whatever your computed value is
- Weak/moderate: salary cap levels the field
- Models achieved 72.9% accuracy without salary, meaning performance is driven by on-field data

Median salary vs actual win%

```
correlation_salary_actual = spearmanr(team_data['median_salary'],  
                                       team_data['actual_win_pct'])
```

Median salary vs predicted win probability

```
correlation_salary_predicted = spearmanr(team_data['median_salary'],  
                                         team_data['predicted_win_pct'])
```

Player	Pos	Age	Cap Hit	Cap Hit Pc	Dead Cap	BaseP5 Sa	Signing Bo	Per Game	RosterBon	OptionBon	WorkoutB	Restructur	Incentives	Team
Kyler Murr	QB	27	49118177	0.1923	-1.1E+08	37000000	3518177	400000	-	7200000	1000000	-	-	ARI Cardinals
Budda Bak	S	28	22086250	0.0865	-1.9E+07	14200000	5061250	-	-	1775000	400000	150000	500000	ARI Cardinals
Jalen Thor	SS	26	12467750	0.0488	-1.2E+07	8300000	2034000	150000	-	1983750	-	-	-	ARI Cardinals
Zaven Coll	ED	25	6876036	0.0269	-1.3E+07	2900000	3710956	255000	-	-	10080	-	-	ARI Cardinals
Kyzir Whit	LB	28	6760080	0.0265	-6760080	5000000	1750000	-	-	-	10080	-	-	ARI Cardinals
Marvin Ha	WR	22	6431771	0.0252	-3.5E+07	795000	5636771	-	-	-	-	-	-	ARI Cardinals
Sean Bunt	CB	27	5955000	0.0233	-1.4E+07	3695000	2000000	210000	-	-	50000	-	-	ARI Cardinals
Hjalte Fro	C	28	4133413	0.0162	-7810080	2000000	1883333	240000	-	-	10080	-	-	ARI Cardinals

CONCLUSION / KEY TAKEAWAYS

- Good feature engineering and avoiding data leakage (using `.shift(1)` + time-based split) were more important than model complexity.
- Simpler models with well-designed features can outperform complex models; Logistic Regression beat both Random Forest and KNN.
- Matchup-relative features (team minus opponent stats) significantly improved predictive performance.
- Model evaluation must go beyond accuracy; ranking ability, confusion matrices, and feature importance revealed deeper insights.
- Overall, the project strengthened our understanding of data cleaning, feature engineering, ML comparisons, and building an end-to-end predictive application.

THANKYOU